

Amortization Engine Implementation Walkthrough

How the Go port computes loan amortization: the field-presence dispatch, the period-by-period schedule engine, the backward solvers, and the math — with the DOS lineage each piece preserves.

Scope. The Go amortization package `internal/finance/amortization/` (engine, backward solvers, first-pass, schedule-oracle bisection, types, validation), the API wiring in `internal/api/handlers.go`, and the shared math in `internal/finance/interest/` and `internal/dateutil/`.

Audience. Engineers maintaining or extending the Go port. Every claim is anchored to a `file:line` reference in `internal/` for verification.

Lineage. The Go code is a faithful port of the DOS Pascal engine; its comments cite the Pascal originals (e.g. `AMORTOP.pas:574`). For the original algorithm, see the companion `legacy/code_docs/amortization_dos_walkthrough.pdf`. This document explains the *Go* implementation.

Contents

0 How to read this document

1 Architecture: request to result

1.1 The package map · 1.2 Pointer fields → field presence · 1.3 The status ladder

2 The data model

3 First pass & dispatch

4 Forward schedule generation

4.1 Rate primitives · 4.2 The simple loop · 4.3 The fancy engine · 4.4 Interest models

5 Backward solving

5.1 Closed-form estimates · 5.2 The schedule-oracle bisection · 5.3 The DOS secant port

6 The math primitives

7 Porting notes & gotchas

8 Appendix: call graph & file:line index

0 How to read this document

The Go port reproduces the Per%Sense amortization logic behind a REST API. Like every Per%Sense screen it is a *field-presence solver*: the caller fills in what is known, leaves one field blank, and the engine solves for the blank — a payment, a rate, a loan amount, or a term. The forward direction generates a period-by-period repayment schedule; the backward direction inverts that schedule.

Citation convention

References look like `engine.go:1178` and resolve under `internal/finance/amortization/` unless another path is named (e.g. `interest/math.go:132`, `api/handlers.go:752`). Where the Go code cites a Pascal line, that lineage is noted as (AMORTOP.pas:NNN).

The two functions to anchor on. `engine.go:139` `Amortize(input)` is the package entry — it classifies, solves the unknown, then generates. `engine.go:1178` `generateFancySchedule` is the period-by-period heart; through the backward solvers' use of it as an oracle, almost every result ultimately runs that loop.

1 Architecture: request to result

1.1 The package map

```

POST /api/amortization/calc
  | HandleAmortizationCalc                                api/handlers.go:752
  ▼ decode AmortizationRequest (pointer fields)
build LoanInput (+ any advanced option ⇒ Fancy=true)    api/handlers.go:812 / :1080
  |
  ▼ amortization.Amortize(input)                          engine.go:139
FirstPass → classify term/dates                          firstpass.go:43
ValidateInputs                                          validate.go:42
solve the unknown (payment / rate / amount / term)     backward.go / engine.go
generate schedule: simple | fancy | exact-in-advance    engine.go:816 / 1178 / 1097
  |
  ▼ AmortResult{Schedule, totals, APR, ...}              types.go:175

```

The files: `engine.go` (entry, schedule generators, rate primitives), `backward.go` (closed-form solvers), `fancybisect.go` (schedule-oracle bisection + the DOS secant port), `firstpass.go` (term/date classification), `types.go` (model), `validate.go`, `advisories.go` (warnings).

1.2 Pointer fields → field presence

The API request uses **pointer types** for every field that participates in field-presence dispatch, so a missing JSON key is distinguishable from a supplied zero `api/handlers.go:180–182`:

```

Amount *float64 `json:"amount,omitempty"`
Rate   *float64 `json:"rate,omitempty"`

```

The handler only stamps `InOutInput` when the pointer is non-nil `api/handlers.go:857–864`; a nil pointer leaves the status at Go's zero value `StatusEmpty` (`=0`). A blank `Payment` arrives as a JSON zero rather than a pointer, so it is explicitly downgraded to `StatusEmpty` `api/handlers.go:902–904`. Supplying any advanced option flips `input.Fancy = true` `api/handlers.go:1080–1082`.

1.3 The status ladder

Each field carries an `int8` status forming the dispatch ladder `interest/constants.go:53–59`: `InOutBad(-1) < StatusEmpty(0) < InOutOutput(1) < InOutDefault(2) < InOutInput(3)`. The pervasive guard `Status ≥ InOutDefault` means “known” (a default or a user input); `Status < InOutDefault` means “blank — solve for it.” A solved field is written back as `InOutOutput` (or promoted to `InOutInput` before a forward run).

2 The data model

The input bundle is `LoanInput` `types.go:145–155`: a `Loan`, the advanced-option slices, `Settings`, and the `Fancy` flag. The core loan `types.go:43–66` pairs each value with a status:

Field	Meaning
<code>Amount</code>	loan principal
<code>LoanDate / FirstDate</code>	closing date / first regular payment date
<code>LoanRate</code>	nominal rate as a yield at <code>PerYr</code> (the engine converts to a continuous rate)
<code>NPeriods / LastDate</code>	term as a count or an end date (mutually exclusive; <code>LastOK</code> tracks which is known)
<code>PerYr / PayAmt / Points</code>	payments per year / regular payment / APR points

Advanced options `types.go:70–125`: `BalloonPayment` (date + amount; a date-only balloon is a target to solve), `RateAdjustment` (ARM: date + optional new rate and/or payment), `Prepayment` (extra periodic stream with a start, a count or stop-date bound, a frequency, and a payment), `Moratorium` (interest-only until `FirstRepay`), `Target` (minimum principal reduction per payment), and `SkipMonths` (a string like "6-8,12" parsed into a 13-element month set). The output is `AmortResult` `types.go:175–193`: the `Schedule` (one `PaymentRecord` per row — date, payment, interest, remaining principal, cumulative interest), totals, optional APR, the echoed post-`FirstPass` term/dates, and resolved balloons.

3 First pass & dispatch

FirstPass [firstpass.go:43–121](#) derives whichever of {first date, last date, #periods} is blank but computable, in three field-presence arms (no iteration): default the first payment date from the loan date + frequency (Amortize.pas: 184); derive lastDate = AddNPeriods(firstDate, perYr, n-1) when first + count are known; or derive n = round(yearsDif(last, first) · perYr) + 1 when first + last are known. The count is computed on a **forced 360 basis** [firstpass.go:104–105](#) to match the DOS NumberOfInstallments. LastOK records whether the term is now known — the central flag the dispatch reads.

Amortize [engine.go:139](#) then validates (≥ 2 payments, ValidateInputs), forces fancy mode where the simple path cannot cope (exact-daily, or USA-rule on exact/non-360 — [engine.go:227–240](#)), determines the unknown and solves it, and branches to a generator:

Unknown (blank field)	Solver	Line
Term (payment known, term blank)	solveFancyTermFromPayment / solveNPeriodsFromPayment	engine.go:261 / 273
Payment (blank PayAmt)	estimatePayment → refine (solveFancyPayment / dosIteratePayment)	engine.go:294, 344, 503–554
Amount / Rate	solved in the API handler before Amortize runs	api/handlers.go:1124 / 1135

The amount/rate solves run in the handler on a throwaway FirstPass'd copy (so the solver guards see derived dates), then write the solved value back as InOutInput so the subsequent Amortize runs forward normally [api/handlers.go:1094–1135](#). The generator branch [engine.go:331](#) is the forward/backward fork: forward = generate; backward = solve then generate.

4 Forward schedule generation

4.1 Rate primitives

All generators share: `GrowthPerPeriod` [engine.go:29–38](#) returns the per-period factor $f = 1 + \text{rate}/\text{RealPerYr}(\dots)$ (weekly/biweekly use an explicit 7- or 14-day fraction); `ComputeTrueRate` [engine.go:44](#) derives the continuous rate for daily accrual; `firstPeriodProrate` [engine.go:753](#) measures the odd first period as a fraction of a normal period (month-based on a clean boundary so the 365 basis doesn't skew a calendar-natural first period); `periodYearFraction` [engine.go:779](#) is the interior-period analogue; `exactDaily` [engine.go:20](#) gates true daily accrual on `Exact && Basis != 360` (AMORTOP.pas:625).

4.2 The simple loop

`generateSimpleSchedule` [engine.go:816–1056](#) handles plain (non-fancy) loans. A user-supplied (hard) payment triggers the “Dav Holle provision”: per-period interest is rounded to cents [engine.go:833](#) (Amortize.pas:1483). The first-period window emits a settlement stub for prepaid loans or prorates the first period's interest. The per-period order of operations [engine.go:930–1052](#):

```
interest = p·(f−1) (interior, arrears) · interest = p·(f−1)·prorate (odd first)
daily: interest = p·(etruerate·YearsDif−1) · in-advance: interest = (p−pmt)·(f−1)/(2−f)
new balance: p = p + interest − pmt
```

Rule-of-78 (360 basis, non-in-advance) front-loads interest via a constant step $r78step = (n \cdot \text{payment} - \text{amount}) / (0.5 \cdot n \cdot (n+1))$ decremented each period [engine.go:920–944](#). On the final period or when the balance would go non-positive, the residual is folded into the payment [engine.go:1016–1021](#).

`generateExactInAdvanceSchedule` [engine.go:1097](#) handles DOS's distinct exact-in-advance shape (a row-0 settlement row, a one-period base shift, then $n-1$ actual-day rows) (AMORTOP.pas:1159–1177).

4.3 The fancy engine

`generateFancySchedule` [engine.go:1178–1812](#) is the period-by-period engine for any advanced-option loan, with a USA-rule exempt-principal bucket usap. `veryLast` [engine.go:1205](#) sets the schedule terminus (latest of last date, balloons, prepay stops). The main loop mirrors DOS `ComputeNext` (AMORTOP.pas:574–664):

1. **Off-cycle draining** [engine.go:1327–1454](#): any balloon/prepayment dated *before* the regular date is emitted as its own dated row with partial-period interest from `prevDate` — applied at its exact date, not folded into the next payment.
2. **Period interest** [engine.go:1456](#) on $(p - \text{usap})$, rounded if hard.
3. **Skip-month** zeroing [engine.go:1477](#), then **moratorium** (interest-only, re-solving the payment at `FirstRepay` only if it was blank) [engine.go:1494](#).
4. **Coincident extras** (balloons + prepayments on this date; `PlusRegular` adds vs replaces) [engine.go:1520](#).
5. **Target**: bump payment to meet the minimum principal reduction — overriding skip-month zeroing [engine.go:1591](#) (AMORTOP.pas:643).
6. **New balance + USA bucket** [engine.go:1653](#), then **rate adjustments** at ARM dates (re-amortize remaining balance, or solve the implied rate from a new payment) [engine.go:1700–1807](#).

4.4 Interest models in one place

Model	Per-period interest	Where
Arrears (simple/compound)	$p \cdot (f-1)$, or actual-day $p \cdot \text{rate} \cdot \text{YearsDif}$	engine.go:967, 996
Daily (continuous)	$p \cdot (e^{\text{truerate} \cdot t} - 1)$	engine.go:975
In-advance (annuity-due)	$(p - \text{pmt}) \cdot (f-1) / (2-f)$	engine.go:945
Rule of 78	decrement by $r78\text{step}$ each period	engine.go:937
USA rule	accrue on $(p - \text{usap})$; bucket floored at 0	engine.go:1654–1659

5 Backward solving

5.1 Closed-form estimates

Each solver starts from annuity algebra. `SolvePayment` [backward.go:70](#): $\text{pay} = \text{amount} \cdot (f-1)/(1-f^{-n})$ via `Lnn/Exxp`, adjusted for first-period proration and in-advance. `SolveLoanAmount` [backward.go:184](#): $\text{amount} = (1-f^{-n})/(f-1) \cdot d + \sum \text{balloon} \cdot e^{-\text{rate} \cdot \text{yrs}}$. `SolveRate` [backward.go:284](#): Newton/secant on the `RepayLoan` residual, first guess $\text{payAmt} \cdot \text{perYr} / \text{amount}$ floored at 0.02, ≤ 30 iterations to $|\text{step}| < \text{teeny}$. `solveNPeriodsFromPayment` [backward.go:365](#): $n = \text{round}(1.4999 + \ln(1-p1(1-ff)/(ff \cdot d))/\ln(ff))$ (`AMORTOP.pas:1382`).

5.2 The schedule-oracle bisection

For fancy loans the closed form is only an estimate; refinement runs against the *real* forward schedule. Because the engine forces the final payment and stops early, the unforced residual isn't directly observable and Newton on it is discontinuous — so `fancybisection.go` uses **bisection** [fancybisection.go:10–26](#). `fancyOverUnder` [fancybisection.go:166](#) runs `Amortize` and classifies a trial as under-amortized (+1), over-amortized (−1), or solved (0), reconstructing the residual as `FinalPrinc + (lastPay - d)`. `fancyBisect` [fancybisection.go:206](#) expands the bracket until the sign straddles, then bisection to a tolerance (5e-4 for amount/payment, 1e-7 for rate). `solveFancyAmount/Rate/Payment` [fancybisection.go:264–320](#) wrap it with the unknown pinned (rate capped at 200%).

5.3 The DOS secant port

`dosIteratePayment` [fancybisection.go:114–157](#) is a bug-for-bug port of `DOS Iterate` (`AMORTOP.pas:1415`), driving `repayExactTerminal` (the unforced terminal balance) to zero by finite-difference secant $\text{newdelta} = \text{delta} \cdot p / (\text{final} - p)$, with the DOS divergence brake (`count += 5` when the step stops shrinking) and the faithful `bestx-after-update` ordering. Convergence: $|\text{residual}| < \text{halfpenny} = 0.005$ or 20 iterations.

The structural heart of the backward path. The closed-form estimate gets the solver close; then the engine is used as its own forward oracle — `fancyOverUnder` calls `Amortize` recursively — and bisection/secant drives the real schedule's terminal balance to zero, the same criterion as DOS's `Iterate`.

6 The math primitives

The shared `interest` and `dateutil` packages reproduce the DOS primitives exactly — the financial behaviour is defined by these, not by the standard library.

Primitive	Behaviour	Location
<code>Round2(x)</code>	round-half- down : halfpenny = 0.005 – teeny, add toward zero then truncate at 2 decimals. The –teeny sends an exact half-cent downward.	<code>interest/math.go:132</code>
<code>Exxp(x)</code>	safe e^x : overflow error at $x > 70$, $1e-32$ below -70 , cubic Taylor for $ x < 1e-4$.	<code>interest/math.go:43</code>
<code>Lnn(x) / Power(x,n)</code>	safe $\ln$ (Taylor near 1, for the Turbo precision bug) / $e^{n \cdot \ln x}$.	<code>interest/math.go:67 / 100</code>
<code>RealPerYr / YieldFromRate / RateFromYield</code>	periods-per-year; continuous ↔ periodic-yield conversions $n(e^{r/n}-1)$ and $n \cdot \ln(1+y/n)$.	<code>interest/rates.go:42 / 63 / 79</code>
<code>YearsDif</code>	day-count years: 30/360 with end-of-month corrections, or actual 365/366 for loan calcs.	<code>dateutil/dateutil.go:632</code>
<code>AddPeriod / AddNPeriods / NumberOfInstallments / Julian</code>	date stepping by frequency; period counting; serial-date math (epoch 1900).	<code>dateutil/dateutil.go:434 / 518 / 709 / 126</code>

7 Porting notes & gotchas

- **Exact requires a non-360 basis.** `exactDaily` is true only when both the `Exact` flag *and* a non-360 basis are set `engine.go:13–22`; on 360, `Exact` is a no-op (`AMORTOP.pas:625`).
- **Exact and USA-rule force fancy mode** `engine.go:227–240` — DOS routes every non-360 loan and every USA-rule loan through the period-by-period engine (`Amortize.pas:1493`).
- **Odd-first payment is augmented, not just prorated.** The closed-form payment shortcut applies only to plain 360/prepaid/in-arrears/natural-first loans; otherwise the engine iterates (`needPaymentRefine`) `engine.go:300–319, 717–731`. The schedule accrues whole-month interest while the payment is augmented — a deliberate DOS asymmetry, documented inline.
- **In-advance blank-payment loans iterate** via the annuity-due secant rather than the ordinary-annuity closed form (`needRefine`) `engine.go:539–553`.
- **Off-cycle balloons land at their exact date** with partial interest, not folded into the next payment `engine.go:1354–1372` (`AMORTOP.pas:608`).
- **Negative implied ARM rate is allowed** (overpaying payment), bounded $|rate| < 2$ — clamping at zero stalled the secant `backward.go:1017–1027`.
- **The fancyBisect-over-Newton choice is intentional** — the forced final payment makes the residual a step function; see the rationale block `fancybisect.go:10–26`.

8 Appendix: call graph & file:line index

```

POST /api/amortization/calc → HandleAmortizationCalc      api/handlers.go:752
├─ decode (pointer fields → status)                      api/handlers.go:857
├─ [one of Amount/Rate nil] backward solve on FirstPass'd copy
│   └─ SolveLoanAmount → solveFancyAmount                backward.go:184 / fancybisect.go:264
│   └─ SolveRate      → solveFancyRate                  backward.go:284 / fancybisect.go:282
├─ Amortize(input)                                       engine.go:139
│   └─ FirstPass (defFirst / last / n)                   firstpass.go:43
│   └─ ValidateInputs                                   validate.go:42
│   └─ GrowthPerPeriod / ComputeTrueRate                 engine.go:29 / 44
│   └─ [term unknown] solveNPeriodsFromPayment | solveFancyTermFromPayment backward.go:365 / 924
│   └─ [payment blank] estimatePayment → solveFancyPayment | dosIteratePayment engine.go:294 / fancybisect.go:166
│   └─ DISPATCH:
│       └─ !Fancy → generateSimpleSchedule                engine.go:816
│       └─ Fancy → generateFancySchedule                  engine.go:1178
│           └─ exact-in-advance → generateExactInAdvanceSchedule engine.go:1097
│   └─ [Points] ComputeAPRWithPoints                     backward.go:439
├─ appendResultAdvisories                                advisories.go:16

```

oracle: every solveFancy*/dosIteratePayment re-runs Amortize via fancyOverUnder / repayExactTerminal
fancybisect.go:166 / 43

Symbol	Reference
Amortize (entry)	engine.go:139
FirstPass	firstpass.go:43
generateSimpleSchedule / generateFancySchedule / exact-in-advance	engine.go:816 / 1178 / 1097
SolvePayment / SolveLoanAmount / SolveRate	backward.go:70 / 184 / 284
fancyBisect / dosIteratePayment	fancybisect.go:206 / 114
Round2 / Exxp / Lnn	interest/math.go:132 / 43 / 67
YearsDif / AddPeriod	dateutil/dateutil.go:632 / 434
API handler	api/handlers.go:752

Per%Sense Go port. Citations resolve under internal/. For the original DOS algorithm this engine ports, see legacy/code_docs/amortization_dos_walkthrough.pdf. Companion Go walk-throughs: mortgage, present value, actuarial.